

Production Log

Project idea

I started with a broad idea about using machine learning to predict cryptocurrency. After looking at the possible directions, I decided that this was too wide for one EPQ. I narrowed the topic to Bitcoin volatility forecasting and used the question:

To what extent can machine-learning models improve Bitcoin volatility forecasting compared with traditional statistical models?

I chose volatility because it is linked to market risk and can be measured and compared more clearly than a general question about whether Bitcoin will go up or down. My main aim was to find out whether the extra complexity of Random Forest and LSTM was actually useful when compared with simpler statistical methods.

Planning and research

At the beginning, I read about log returns, volatility, rolling historical volatility, GARCH, Random Forest and LSTM. I planned to compare the models using the same Bitcoin data and the same test period. Accuracy was important, but I also wanted to consider how easy each model was to explain, how long it took to run and whether the result was stable.

My first plan used Yahoo Finance data. I later changed to Hyperliquid because it gave me daily data from one clearly defined Bitcoin perpetual-futures market. The data included the open, high, low, close, volume and trade count for each day.

I planned the project in this order:

1. collect and check the Bitcoin data;
2. calculate log returns and a rolling volatility target;
3. build simple baseline methods;
4. add Random Forest and LSTM;
5. compare every model on later data;
6. check the results again using different settings;
7. use the results to answer whether extra complexity was worthwhile.

Data preparation and first comparison

I downloaded the daily Hyperliquid data and kept the original rows before creating the processed dataset. I calculated daily log returns and used a 30-day rolling window to estimate volatility. The target for each row was the next day's 30-day volatility value.

I started with rolling historical volatility because it was simple and easy to explain. I then added GARCH because it was designed to model periods of high and low volatility. I also added lagged linear regression as a simple check before using Random Forest and LSTM.

I kept the data in date order. The earlier part was used for training and the later part for testing. I did not randomly shuffle the rows because that could allow future market information to affect an earlier forecast. I used MAE, MSE and RMSE to compare the size of the prediction errors.

Problems and changes

The project changed several times after I checked the data and outputs more carefully.

The first important issue was an unfinished daily candle. A data refresh included the current day before that day had ended, so the closing price, volume and trade count were incomplete. I changed the data step so that only completed daily candles were kept.

The second issue was the date alignment of the GARCH predictions. Some predictions had been connected by row number rather than by the real date. This meant that a prediction could be compared with the wrong target day. I changed the matching to use dates and then reran the full comparison.

I also corrected the GARCH update order and checked the rolling-volatility calculation. For the LSTM, I made sure that scaling was based only on the earlier training data. These changes affected some error values, so I updated the results and discussion instead of keeping the earlier ranking.

Further checks

I did not want the conclusion to depend on only one test. I repeated the comparison using both 14-day and 30-day volatility targets. I also checked the first and second halves of the test period, low-, medium- and high-volatility periods and four expanding time windows.

For the machine-learning models, I checked Random Forest feature importance and ran the LSTM with three different random seeds. The exact results changed slightly, but the main conclusion stayed the same.

Final result

In the final refresh, Hyperliquid returned 1,241 daily rows. I removed one unfinished row and kept 1,240 completed days through `2026-07-19`. After preparing the data, I used 950 earlier rows for training and 245 later rows for testing.

The final 30-day RMSE results were:

Model	RMSE
GARCH(1,1)	0.00098502
Lagged linear regression	0.00140087
Rolling historical volatility	0.00142744
LSTM	0.00174351
Random Forest	0.00232370

GARCH ranked first in the main comparison and also remained first in the 14-day and expanding-window checks. Neither LSTM nor Random Forest beat the rolling baseline overall.

My answer to the project question is therefore that the tested machine-learning models did not improve accuracy enough to justify their extra complexity in this project. This conclusion only applies to the data, period and model settings I used; it does not prove that machine learning will always perform worse for Bitcoin.

Reflection

The most useful part of the project was not simply running more models. It was finding and correcting problems in the data and dates. Before this project, I was more likely to trust a result if the output looked reasonable. I now understand that I need to check what the data represents, when it became available and whether every prediction is compared with the correct target.

I also learned that a simpler model can be a strong result rather than a disappointing one. My original expectation was that at least one machine-learning model would perform best, but the final evidence did not support that. Keeping the conclusion different from my first expectation made the project more honest.

If I repeated the project, I would keep a short weekly record from the beginning, decide all comparison rules before running the models and reserve a completely untouched final test period. I would also consider using higher-frequency data or comparing a second market, but only after the main process was working correctly.